Shrinkage improves predictive AI evaluation under benchmark holdout: a reproducible study and public-data audit

Francisco Angulo de Lafuente
Independent researcher, Madrid, Spain
lareliquia.angulo@gmail.com

October 24, 2026

Abstract

We study predictive evaluation of AI systems under the benchmark-level holdout defined by the Predictive AI Evaluation Competition, where the items to be predicted come from benchmarks absent from training. From the public `measurement-db` release we build a local simulation of the competition's Brier ALC, validate it against hand-constructed cases, and evaluate under leave-one-benchmark-out over 101 held-out public benchmarks that were not inspected during development. Our submission is the posterior mean of a Beta–Binomial model with a symmetric prior and a single shrinkage strength, taken as the argmin of a disjoint 50-benchmark development split. It reduces Brier ALC from 0.2454 to 0.2105 against the official empirical-mean reference: an absolute reduction of 0.0349 and a relative reduction of 14.2%. On a run with a randomised acquisition order, resampling benchmarks with their pairs kept together gives -0.0310, 95% CI $[-0.0373, -0.0249]$. The gain improves 86 of 101 benchmarks, survives dropping the five benchmarks where it is largest, holds between 13.1% and 14.5% across every sampling choice we varied, and falls only to -0.0323 when benchmarks are reweighted by their eligible pair counts. All of these figures come from our own approximation of the evaluation, not from the official scorer, and the reference they beat is deliberately simple. We then report seven extensions that produced no detectable improvement, including two direct attempts at the largest remaining target: an oracle restricted to one value per pair scores 0.1789, and 96.7% of that floor is variation *between items inside a pair*, yet models of per-item deviation fitted out-of-benchmark — with hand-built features and with hashed sparse text — made results strictly worse at every weight we tried, even when started from the true pair rate. Finally we audit the release and report three quantified findings. All code, the harness and its ten tests are released.

1 Task and scoring

A *subject* is an AI system fixed by model, harness, harness version and reasoning effort. An *item* is a task. A *response* is one scored run, 1 if correct. The predictor receives the subject attributes, the item attributes (content, features, interactors, an anonymous `benchmark_id`) and whatever labels the current budget has revealed, and returns $\hat{p} \in [0, 1]$ without running the subject.

Two properties of the scoring rule drive what follows.

The unit of scoring is the subject–benchmark pair. Brier ALC is computed per pair and the pairs are averaged with equal weight, regardless of how many responses each contributes:

$$\mathrm{ALC} = 0.1\,B_0 + 0.2\,B_1 + 0.2\,B_3 + 0.2\,B_7 + 0.2\,B_{15} + 0.1\,B_{31}, \tag{1}$$

where B_n is the Brier score at a budget of n revealed labels for that pair. Section 7 shows that matching a prior's averaging to Equation (1) mattered more, in our experiments, than any modelling refinement we tried.

The holdout is at benchmark level. Test items are drawn from benchmarks assigned to a private pool, so benchmark-specific parameters cannot be transferred. Whether *item* properties transfer is an empirical question; we test it twice in Section 7, for absolute difficulty and for within-pair relative difficulty. We validate by holding out whole benchmarks and never by splitting responses at random: 68.8M observations are not 68.8M independent examples of transfer.

2 Data

The public release contains 249 curated benchmarks. Of the 281 catalogue records we read, 278 are marked `release == public` (three are withheld for source reproducibility) and 165 of those are binary, contributing 68,840,488 recorded responses over 14,733 subject–benchmark rows, of which 13,289 have at least 80 responses. We use the `responses`, `items`, `subjects` and `benchmarks` tables; `traces` and `assets` (244 GiB together) are never downloaded because the predictor is never shown them. Everything in this paper concerns those 165 binary benchmarks, not the release as a whole.

Local evaluation set. We follow the published construction: pairs with at least 80 distinct items, items split 50/50 into an acquisition pool and an evaluation pool with a fixed per-pair seed, acquisition items never scored, evaluation outcomes never revealed, nested label sets at budgets $\{0, 1, 3, 7, 15, 31\}$, and one `predict` call per distinct evaluation item whose value is reused by every recorded response of that item. Two choices are ours and not part of the official procedure: we sample at most six pairs per benchmark and cap each pair at 150 distinct evaluation items. Section 6 measures how much they matter. The result is 769 pairs over 151 benchmarks and 4.78M scored responses.

Development and headline splits. From a fixed seed we assigned 50 benchmarks to development and 101 to the headline set (496 pairs). The shrinkage strength and the choice of submitted method were fixed on development. Section 9 states exactly which analyses touched the headline set and when.

3 Harness validation

We have neither the official scorer nor the private data, so we do not claim numerical agreement with the official score. We claim that our implementation satisfies the published definition, and we test it. Ten checks ship with the code (`src/test_metrica.py`), covering the weights, the constant-0.5 and perfect-predictor limits, pool disjointness, nesting, the 80-item threshold, equal pair weighting, reuse of one prediction across repeated responses, and the hand-built case where unsmoothed averaging scores 0.925. All ten pass.

All partitions are derived with `blake2b`, not Python's built-in `hash`, which is randomised per process. An earlier version of this work used the latter; an independent re-derivation caught it, and every number below was regenerated afterwards. We mention it because it is the kind of defect that silently makes a reproducibility claim false.

4 Method

Our submission is the posterior mean of a Beta–Binomial model for the pair:

$$\hat{p} \;=\; \frac{\alpha + k}{\alpha + \beta + n}, \qquad \alpha = P_0 S, \quad \beta = (1 - P_0) S, \tag{2}$$

with k successes among the n labels revealed for that subject and that `benchmark_id`, $P_0 = 0.5$ and $S = 3$. $S = 3$ is the argmin on development inside a region that is flat over $S \in [3, 5]$ (0.21708, 0.21740, 0.21809). The predictor keeps no state, fits nothing at run time, and needs no network, no files and no payload. If `benchmark_id` is ever absent it returns the prior rather than pooling labels across benchmarks.

The official reference instead returns the raw mean of the revealed labels, and 0.5 when none exist. At $n = 1$ that is 0 or 1, so a single wrong guess costs the maximum Brier loss of 1. This is not hypothetical: on the headline set the reference scores $B_1 = 0.3650$, worse than ignoring the labels entirely, and Table 1 shows that almost the whole gap between the two methods opens at B_1 and B_3.

5 Results

Table 1 gives the headline result: Brier ALC falls from 0.2454 to 0.2105, an absolute reduction of 0.0349 and a relative reduction of 14.2%.

Table 1: Headline set: 101 held-out benchmarks, 496 pairs, default acquisition order (`src/ejecuta.py final`). Lower is better. The three rows nearest the submission are within noise of it; none was chosen after seeing this table.

Predictor	ALC	B_0	B_1	B_3	B_7	B_{15}	B_{31}
Empirical-Bayes prior, subject-conditioned	0.2099	0.2471	0.2311	0.2119	0.1991	0.1905	0.1865
Hierarchical prior, pair-averaged	0.2102	0.2484	0.2313	0.2127	0.1994	0.1905	0.1863
Beta–Binomial, $S{=}3$ (submitted)	**0.2105**	0.2500	0.2319	0.2128	0.1992	0.1905	0.1863
Empirical-Bayes prior	0.2109	0.2512	0.2332	0.2125	0.1992	0.1906	0.1864
Ablation: subject effect only	0.2137	0.2650	0.2368	0.2156	0.1996	0.1906	0.1863
Ablation: global mean only	0.2175	0.2770	0.2450	0.2188	0.2007	0.1912	0.1864
Empirical mean (*official reference*)	0.2454	0.2500	**0.3650**	0.2434	0.2076	0.1925	0.1868
Constant 0.5	0.2500	0.2500	0.2500	0.2500	0.2500	0.2500	0.2500
Oracle, one value per pair	*0.1789*						

Uncertainty. The 496 pairs are not 496 independent units for an inference about new benchmarks, because pairs cluster inside their benchmark. We therefore resample *benchmarks*, keeping their pairs together. On a run with a randomised acquisition order the difference is -0.0310 with a 95% interval of $[-0.0373, -0.0249]$ and no replicate of the opposite sign. Averaging per benchmark rather than per pair gives 0.2088 against 0.2393 on that run; the two units agree closely here, but we report them separately because they are not interchangeable.

Where the gain lives. Back on the default-order run, the method improves 86 of 101 benchmarks and worsens 15. The median per-benchmark difference is -0.0405, the 10th and 90th percentiles are -0.0701 and $+0.0127$, the best case is -0.1273 and the worst $+0.0463$. Dropping the five benchmarks where it gains most moves the mean difference only from -0.0347 to -0.0317, so the result is not carried by a handful of benchmarks.

Stability. Across five random acquisition draws our score ranges over $[0.2096, 0.2110]$ (s.d. 0.0005) and the reference over $[0.2406, 0.2447]$ (s.d. 0.0014).

How much is left. An oracle that knows each pair's accuracy but must emit one value per pair — the best any pair-constant predictor can do, ours and the reference included — scores 0.1789 on the same pairs. The reference leaves 0.0665 of headroom and we capture 0.0349, or 52.5%. Decomposing that floor, 0.0059 (3.3%) is variation between repeated runs of the same item and 0.1730 (96.7%) is variation *between items inside a pair*. The rest of the problem is therefore about which items in an unseen benchmark are hard, and Section 7 reports our failed attempt on exactly that.

A claim we decline to make. It would be convenient to say the official reference is no better than doing nothing. It is better: mean ALC 0.2454 against 0.2500. But the benchmark-cluster interval for that difference, $[-0.0064, +0.0252]$ on the randomised-order run, includes zero; with the default order the gap is $+0.0046$. Our 14.2% relative reduction should be read alongside the absolute difference of 0.0349, and against a reference that is deliberately simple rather than competitive.

6 Sensitivity to our local approximations

Varying the cap on evaluation items per pair over $\{25, 50, 100, 150\}$ gives relative reductions of 14.5%, 14.3%, 14.3% and 14.2%. Varying the number of pairs sampled per benchmark over $\{1, 2, 4, 6\}$ gives 13.1%, 14.3%, 14.2% and 14.2%.

A third approximation is less innocent. Sampling a fixed number of pairs per benchmark gives every benchmark equal weight, whereas the competition averages pairs. Among the headline benchmarks there are $11{,}657$ eligible pairs and the five largest hold 77.9% of them (`alignment_faking` 36.7%, `sib200` 28.7%). Reweighting the per-benchmark differences by eligible pair counts moves the mean difference from -0.0347 to -0.0323: still clearly favourable, but we report it because our proxy does under-weight the benchmarks the official metric would emphasise.

7 Extensions that produced no detectable improvement

Averaging the prior the wrong way cost more than any refinement. Our first hierarchical prior used the response-weighted global accuracy, 0.350, which is dominated by a handful of enormous benchmarks. The quantity Equation (1) averages is the per-pair accuracy, 0.538 among the rows the competition evaluates. Using the former raised ALC to 0.2137 and B_0 to 0.2650, against 0.2105 and 0.2500 in the same run. Among the eligible rows the two averages are 0.350 and 0.538; the figure of 0.549 quoted above is over all rows. For this objective, averaging the prior the way the score is averaged was the single most consequential choice we made; we do not offer it as a general rule for conditional priors.

A subject-identity prior does not transfer reliably. Conditioning the prior on the subject's accuracy in other benchmarks improves B_0 from 0.2500 to 0.2471 — a real if small signal — but the ALC difference against the plain symmetric prior is -0.0005 with a 95% interval of $[-0.0024, +0.0012]$ and $p = 0.64$ on the randomised-order run. With the default acquisition order it wins in 46 of 101 benchmarks, sign-flip $p = 0.43$. Among eligible rows only 16 subjects appear in five or more benchmarks, and a subject's accuracy varies by 0.130 across benchmarks against a between-pair spread of 0.273 (`src/estadisticas_datos.py`).

A non-parametric prior over the bimodal distribution does not help. Per-pair accuracy is strongly bimodal. Among the $13{,}289$ rows with at least 80 responses — the only ones the competition evaluates — the modal bin is $[0, 0.05)$, holding 8.3% of them, and a broad second mode covers the upper half; the macro mean is 0.538 with a spread of 0.273. A single Beta cannot represent that shape (`src/estadisticas_datos.py`). A non-parametric prior on a 101-point grid, fitted out-of-benchmark, scores $+0.0001$ against Equation (2) ($p = 0.81$ on the randomised-order run, -0.00002 and $p = 0.55$, 47 of 101 benchmarks, with the default order), and this held across filters on minimum pair size, recentring, and uniform mixing weights from 0.05 to 0.3 (`src/barrido_prior.py`). The experiment does not establish whether the limitation lies in estimating the prior, in transfer across benchmarks, or in the information the revealed labels carry.

Per-budget shrinkage tuning buys nothing. Because the predictor is constant per pair, each budget can be optimised independently. Doing so selects $S = 3$ at every budget with at least one label and reaches 0.21708 on development, identical to a single $S = 3$. At budget 0 there are no labels, so every S gives the same prediction and the sweep is degenerate there.

Cheap item-text features give nothing detectable at budget 0. We fitted a regularised logistic model on 1.17M development observations over 23 columns: twelve item-text features computable inside `predict` — length, word count, digit, uppercase, non-ASCII and punctuation ratios, code and LaTeX markers, option count, multiple-choice indicator, question marks, line breaks — plus release year, harness and provider indicators. Under leave-one-benchmark-out *within development* it scores 0.2495 against 0.2500 for the constant, winning in 18 of 50 benchmarks ($p = 0.065$), worth -0.00005 in ALC. Several coefficients have plausible signs (multiple-choice easier, code and length harder), but coefficient signs alone do not establish transferable signal.

Per-item prediction, the largest target, made things worse. Since 96.7% of the remaining floor is between-item variation and `predict` receives the specific item, a pair-constant predictor leaves that entire share untouched by construction. The previous experiment asked whether features predict *absolute* correctness, which conflates pair difficulty with item difficulty. We therefore asked the sharper question: do they predict an item's *deviation from its own pair's mean*? We fitted a ridge model on 1.63M development observations with the pair mean as an offset, so the coefficients capture only within-pair relative difficulty, and added its prediction to Equation (2) with weight λ. Development ALC was 0.21708 at $\lambda = 0$, matching Equation (2) exactly, and rose monotonically to 0.21776, 0.21896 and 0.22067 at $\lambda = 0.5, 1.0, 1.5$. Adding this signal is strictly harmful at every weight we tried.

Nor does a far richer text representation. To bound that result beyond one hand-built feature set, we repeated it with hashed sparse text features — 2^{18} word uni/bigrams plus 2^{17} character 3–4-grams, fitted with SGD — and made the test as generous as possible by starting from the *true* pair mean rather than from our estimate. Over 50 development benchmarks the oracle pair mean scores 0.1905; adding the predicted deviation gives 0.1907 at $\lambda = 0.5$ and 0.1912

at $\lambda = 1.0$, improving only 14 of 50 benchmarks (sign-flip $p = 0.003$ *against* the addition). The mean correlation between predicted and realised deviation is -0.016, with a maximum of 0.196 on a single benchmark. So even given the pair rate for free, cheap text features do not identify which items an unseen benchmark's subject will miss.

A route we could not test: the organisers' companion embedding release `measurement-db-embed` covers only 3 of the 165 binary benchmarks, which is too few to measure transfer across benchmarks. A learned representation from a language model remains the obvious untried candidate.

Why we submit no acquisition function. The competition allows an optional `labeling.py` to choose which outcomes to reveal. We submit none, and we want to be careful about why. Our predictor cannot distinguish items, so it has no basis on which to prefer one candidate over another, and we fall back to the platform's random default. We do *not* claim random is optimal: since 96.7% of the pair-constant floor is between-item variance, a policy that selected candidates by content could in principle estimate the pair rate with less variance than a random draw. That is an untested opportunity we leave open, and it rests on the same item-level signal the previous experiment failed to find.

Taken together, in this protocol and on this release, the improvement over the official reference comes from calibrating how far a few revealed labels may move the estimate, and seven attempts to add information beyond that produced nothing detectable. This bounds what we extracted with cheap, fully offline methods. Item embeddings, inference-time model calls and richer text representations remain untested, and the 0.1730 of between-item variance remains available to whoever can reach it.

8 Audit of the public release

Definitions were fixed before looking at the data: "same item" means an identical published `content_hash`; "repetition" means two response rows with the same subject and item; a "post-response field" describes how an answer is graded rather than what the system was shown. Nothing here uses platform feedback, and we made no attempt to identify the subjects or benchmarks behind anonymous identifiers.

Confirmed: items are shared across benchmarks. Of $2{,}898{,}062$ distinct content hashes over $3{,}770{,}701$ items in the 165 binary benchmarks, $1{,}980$ (0.068%) appear in more than one benchmark, involving 11 benchmarks. The largest overlaps are `adaptivestep`↔`prox` ($1{,}819$ items), `adaptivestep`↔`felm` and `felm`↔`prox` (215 each), and `swe_rebench`↔`swebench_live` (91). This is irrelevant to our submitted predictor, which never reads item content, but not in general: any method using item text could see the same item in two benchmarks and count it as independent evidence. The case that would matter most — whether any such item also sits in a private test benchmark — cannot be checked from outside, and we do not claim it does. We raise it as a question for the organisers, who can.

Not found: leakage into predictor-visible fields. We screened the fields the predictor actually receives for answer-, solution-, label- and grading-like content across 60 of the 165 benchmarks. Two had keyword matches in `item_features`, and in neither case did the field coincide with the benchmark's grading criterion. We record this as no finding in a keyword screen over 60 benchmarks, which is not evidence of absence.

Confirmed: a fifth of repeated observations contain both outcomes. Across 87 benchmarks with repeated measurements there are $3{,}154{,}268$ subject–item pairs measured more than once, and $635{,}977$ of them (20.2%; median 8.3% per benchmark) contain both a success and a failure. Some pairs are repeated up to $10{,}000$ times. This need not indicate a grader fault — stochastic sampling by the system explains much of it — and it does *not* by itself explain the 0.1789 pair-constant oracle score: as measured in Section 6, repetition variance accounts for 0.0059 of that figure and between-item variance for 0.1730. What it does mean is that a measurable share of response-level Brier loss is irreducible for any predictor, and that should be stated whenever a Brier number on this data is interpreted.

9 Limitations

Our local score is not the official score. We approximate the evaluation with our own harness, cap evaluation items at 150 per pair, sample at most six pairs per benchmark, restrict to binary benchmarks, and weight benchmarks rather than

pairs (Section 6 quantifies the last three). We report Brier ALC on public training benchmarks held out from each other, which is a proxy for, not a measurement of, performance on the private pool.

On development discipline we want to be precise rather than flattering. The shrinkage strength is the development argmin, written by `src/barrido_prior.py`, and we did not select it by looking at the headline set. We did, however, score the headline set twice with two values of it. An earlier version of this work fixed $S = 4$ by hand inside the flat region and reported 0.2111 and a 14.0% reduction; we then replaced it with the development argmin $S = 3$, which gives 0.2105 and 14.2%. The change was made to remove an arbitrary choice, not because the headline improved, and it moves the result by 0.0006. We state it because a reader cannot otherwise tell from the artefacts that two headline runs exist. Separately, the headline set was not scored exactly once: it was scored by the final run, then again under five acquisition draws and $2{,}000$ bootstrap replicates, and the sensitivity study re-scored it under other sampling choices. The per-budget sweep, the two text experiments and the sensitivity study were all designed after the headline numbers existed; all are reported on development or as sensitivity around an already-fixed result, and none changed the submission, but they are exploratory rather than pre-registered. The audit statistics and the bimodality figures are computed over all 165 binary benchmarks, headline ones included, since they describe the data rather than evaluate a method. An intermediate version of the headline table was also discussed with a language model used as a critic during development; that discussion changed the framing and the analyses, not the submitted predictor.

Our negative results are bounded by what we tried, with cheap offline methods, on this release, under our approximation of the protocol.

10 Reproducibility

Everything in this paper is produced by a named script. The data are the public `measurement-db` release, fetched with `src/descarga_datos.py`, at verified data commit

`a23e041be8651be4d97decba385779300abeabee`

and builder revision

`287495967fc276275c3b466dfede2cc3534e830c`.

The evaluation set is built by `src/construye.py` with master seed 20261024, and the development/headline partition is drawn from the same seed in `src/ejecuta.py`. Every result maps to one script:

Result	Script
Ten harness checks	`test_metrica.py`
Descriptive statistics	`estadisticas_datos.py`
Submitted shrinkage strength (writes `hiper.json`)	`barrido_prior.py`
Per-budget sweep	`barrido_presupuesto.py`
Hierarchical-variant curve	`ejecuta.py tune`
Table 1	`ejecuta.py final`
Uncertainty, stability, both averaging units	`robustez.py`
Sensitivity, oracle decomposition, reweighting	`sensibilidad.py`
Audit	`auditoria.py`
Text experiments	`texto_b0.py`, `por_item.py`

The `tune` mode sweeps the hierarchical variants only and does not write `hiper.json`. The submitted strength is reproduced by running `barrido_prior.py`, the sole writer of that file. Requirements are Python 3.10+, `pandas`, `pyarrow`, `numpy`, `scikit-learn` and `huggingface_hub`, about 14 GiB of disk, and a few hours on one desktop CPU with no GPU. The submitted `model.py` is standard library only, carries no payload and depends on nothing at run time.

Disclosure. All training and development used only the public `measurement-db` release listed in the benchmark inventory; no other data source was used. The work was carried out with substantial assistance from large language models, for code, for prose and as adversarial critics of the analysis. All numerical results were produced by the released scripts, and an independent re-derivation — which found and forced the correction of the non-deterministic partitioning described in Section 3, among other errors — was run before submission.